Chapter 3

Algorithmic Number Theory

Throughout this chapter: let p and q be prime numbers, and let $m \geq 2$ be any prime or composite number. The set of all integers is denoted $\mathbb{Z}$.

3.1 Modular arithmetic background

3.1.1 Divisibility

For any integers $a \in \mathbb{Z}$ and $m \geq 2$, there exist unique integers $k \in \mathbb{Z}$ and $0 \leq r < m$ such that

$$a = km + r.$$

In particular,

$$k = \lfloor a/m \rfloor, \quad r = a - km.$$

The value r is called the *remainder*. If $r = 0$, then m *divides* a, which is denoted $m \mid a$.

Fact 3.1 (Euclid's Lemma)**.** *If $p \mid ab$ and p is prime, then $p \mid a$ or $p \mid b$.*

The *greatest common divisor* of two integers a, b, denoted $\gcd(a, b)$, is the largest nonnegative integer x such that $x \mid a$ and $x \mid b$. (By convention, we set $\gcd(0, 0) = 0$.)

Fact 3.2 (Euclid's algorithm)**.** *There exists a PPT algorithm for computing* $\gcd(a, b)$.

More precisely, a deterministic algorithm computes gcd in time polynomial in the total binary input length, without knowing the prime factorization of a or b.

3.1.2 Modular arithmetic ($\mathbb{Z}_m$)

Given a *modulus* $m \geq 2$, modular arithmetic identifies a number a with its remainder. We write $[a \bmod m]$ for the remainder of a when divided by m. As explained above, it is the unique number in the range $0 \leq r < m$ such that $a = km + r$ for some integer k. The operation $a \mapsto [a \bmod m]$ is called *modular reduction*.

We denote by $\mathbb{Z}_m$ the set of integers $\{0, 1, 2, \ldots, m-1\}$. Integers a and b are *congruent modulo* m if $[a \bmod m] = [b \bmod m]$. We write this as $a \equiv b \bmod m$ or $a \equiv_m b$. Equivalently, $a \equiv b \bmod m$ if and only if either of the following holds:

- $a = mk + b$ for some integer $k \in \mathbb{Z}$,
- $m \mid (a - b)$.

One thing that makes modular arithmetic useful is that it respects integer addition and multiplication.

Proposition 3.1. *If $a \equiv_m c$ and $b \equiv_m d$, then*

- $a + b \equiv_m c + d$, *and*
- $a \cdot b \equiv_m c \cdot d$.

Proof. See Exercise 3.1. □

Modular addition and multiplication are associative and commutative. 0 is the *additive identity* ($a+0 \equiv a \equiv 0+a \bmod m$) and 1 is the *multiplicative identity* ($a \cdot 1 \equiv a \equiv 1 \cdot a \bmod m$). Moreover, every element $a \in \mathbb{Z}_m$ has an *additive inverse* $[-a \bmod m] = [m - a \bmod m]$: $a + (-a) \equiv 0 \bmod m$.

Example 3.1. Let $m = 5$. You can compute $3 + 8 \pmod 5$ in any of the following ways, and many others:

- $3 + 8 \equiv 3 + 3 \equiv 6 \equiv 1 \bmod 5$.
- $3 + 8 \equiv 11 \equiv 1 \bmod 5$
- $3 + 8 \equiv -2 + 3 \equiv 1 \bmod 5$

Likewise, you can compute $7 \cdot 7 \pmod 5$ in any of the following ways, and many others:

- $7 \cdot 7 \equiv 2 \cdot 2 \equiv 4 \bmod 5$
- $7 \cdot 7 \equiv 49 \equiv 4 \bmod 5$
- $7 \cdot 7 \equiv -3 \cdot 2 \equiv -1 \equiv 4 \bmod 5$

3.2 Multiplication and inverses modulo p ($\mathbb{Z}_p^*$)

Modular multiplication can be weird. For example, the product of two non-zero elements can be congruent to 0!

$$6 \cdot 10 \equiv 60 \equiv 0 \bmod 15.$$

This section shows that **multiplication is well-behaved when the modulus is prime**.

Proposition 3.2 (Zero product property mod p)**.** *Let p be prime. If $ab \equiv 0 \bmod p$, then $a \equiv 0 \bmod p$ or $b \equiv 0 \bmod p$.*

Proof. $ab \equiv 0 \bmod p \iff p \mid ab$. By Euclid's Lemma, $p \mid a$ or $p \mid b$. Therefore $a \equiv 0 \bmod p$ or $b \equiv 0 \bmod p$. □

Proposition 3.3 (Cancellation property mod p)**.** *Let p be prime and $a \not\equiv 0 \bmod p$. If $ab \equiv ac \bmod p$, then $b \equiv c \bmod p$.*

Proof. $0 \equiv (ab - ac) \equiv a(b - c) \bmod p$. Therefore $b - c \equiv 0 \bmod p$. □

3.2.1 Multiplicative inverses mod p

In this section, we will show that every non-zero element $a \not\equiv 0 \bmod p$ has a *multiplicative inverse* mod p. Zero doesn't have a multiplicative inverse: there is no element $0^{-1} \in \mathbb{Z}_p$ such that $0 \cdot 0^{-1} \equiv 1 \bmod p$.

We have a special notation for the non-zero elements of $\mathbb{Z}_p$.

$$\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\} = \mathbb{Z}_p \setminus \{0\}.$$

Using this notation, for $a \in \mathbb{Z}_p$, we have $a \in \mathbb{Z}_p^* \iff a \not\equiv 0 \bmod p$.

Theorem 3.1 (Fermat's Little Theorem)
Let p be prime and $a \in \mathbb{Z}_p^*$.

$$a^{p-1} \equiv 1 \bmod p.$$

Proof. All arithmetic below is modulo p. Consider the set $S = \{a \cdot x \bmod p : x \in \mathbb{Z}_p^*\}$, with multiplication done modulo p.

First, we show that $S = \mathbb{Z}_p^*$. Observe that $0 \notin S$ because $a, x \not\equiv 0 \bmod p$ by the zero product property. Therefore $S \subseteq \mathbb{Z}_p^*$. Next, observe that for $x \not\equiv y$, the cancellation rule implies that $ax \not\equiv ay$. Therefore $|S| = |\mathbb{Z}_p^*|$. Hence, $S = \mathbb{Z}_p^*$.

Second, we use the preceding result to prove the theorem. Because $S = \mathbb{Z}_p^*$, the product of all their elements must be the same:

$$\prod_{x \in \mathbb{Z}_p^*} x \equiv \prod_{x \in S} x.$$

Expanding:

$$(p-1)! \equiv 1a \cdot 2a \cdot 3a \cdots (p-1)a \equiv a^{p-1} \cdot (p-1)! \bmod p.$$

Cancelling each of the nonzero terms in $(p-1)!$ using the cancellation rule, we conclude:

$$1 \equiv a^{p-1} \bmod p. \qquad \square$$

The existence of **multiplicative inverses** is an immediate corollary. (Uniqueness of multiplicative inverses follows from the cancellation property.)

Corollary 3.1. *Let p be a prime and $a \in \mathbb{Z}_p^*$. Then $a^{-1} \equiv a^{p-2} \bmod p$ is the multiplicative inverse of a modulo p.*

But just because we have an explicit formula to compute multiplicative inverses doesn't mean we can compute them efficiently. For that, we need to figure out how to compute $[a^{p-2} \bmod p]$ in PPT, which is not as simple as it may seem! The next section shows that **modular exponentiation (and thus multiplicative inverses) can be computed in PPT.**

3.2.2 Modular exponentiation by repeated squaring

We define modular exponentiation just like regular exponentiation, as repeated multiplication. Let's analyze the complexity of computing $f : (a, x, m) \mapsto [a^x \bmod m]$, for $a \in \mathbb{Z}$, an integer $x \geq 0$, and $m \geq 2$. We set $a^0 = 1$, including when $a = 0$. Exponentiation over $\mathbb{Z}$ ($(a, x) \mapsto a^x$) is **not** PPT. This is because the output a^x takes exponentially many bits to write down in the worst case. For example, 2^x takes $x + 1$ bits to represent. As we said above, x is exponential in its bit length $\log x$. Modular exponentiation doesn't have this problem. Its output is in the range $[0, m)$ – values which can be represented in fewer bits than the input itself.

A naive algorithm performs $O(x)$ modular multiplications (each multiplication itself has a polynomial cost in the bit length of m):

$$a^x \equiv \underbrace{a \cdot a \cdot a \ldots a \cdot a}_{x \text{ times}} \mod m.$$

As with the $O(\sqrt{N})$-time algorithm for factoring, this is not polynomially-bounded.

Luckily, there is a polynomial time algorithm for modular exponentiation, called **repeated squaring**. For $x = 0$, return 1. For $x > 0$, let $n = \|x\| = \lfloor \log_2 x \rfloor + 1$.

- Compute the binary representation of x. These are the values $z_0, z_1, \ldots, z_{n-1} \in \{0, 1\}$ such that $x = \sum_{i=0}^{n-1} z_i \cdot 2^i$.
- First reduce a modulo m, then compute the n residues $[a^{2^i} \bmod m]$ for $0 \leq i < n$:

  $$a \equiv a^{2^0} \mod m, \quad a^2 \equiv a^{2^1} \mod m, \quad a^4 \equiv a^{2^2} \mod m, \quad \ldots, \quad a^{2^{n-1}} \mod m.$$

  After the first term, each term can be computed with one multiplication (squaring the previous term) and one modular reduction, reducing values modulo m as you go. Each multiplication only involves two terms smaller than m.
- Compute $a^x \bmod m$ by computing the product

  $$\prod_{i:z_i=1} a^{2^i} \equiv \prod_{i=0}^{n-1} a^{z_i \cdot 2^i} \equiv a^{\sum_{i=0}^{n-1} z_i \cdot 2^i} \equiv a^x \mod m.$$

  Multiplying the selected residues and reducing after each multiplication requires at most $n - 1$ further multiplications and modular reductions.

Each of the three steps in this algorithm requires $O(n)$ poly-time operations, so the whole thing can be done in poly-time in the bit length of a, x, m.

Example 3.2. To compute $3^{13} \bmod 7$, write $13 = 8 + 4 + 1 = 0b1101$. Repeated squaring gives

$$3^1 \equiv 3 \mod 7, \qquad 3^2 \equiv 2 \mod 7, \qquad 3^4 \equiv 4 \mod 7, \qquad 3^8 \equiv 2 \mod 7.$$

Use the powers corresponding to the three 1-bits:

$$3^{13} \equiv 3^8 \cdot 3^4 \cdot 3^1 \equiv 2 \cdot 4 \cdot 3 \equiv 3 \mod 7.$$

3.3 Chinese Remainder Theorem ($\mathbb{Z}_N$)

Let $p \neq q$ be two primes.[1] The Chinese Remainder Theorem (CRT, due to Sun Zi aka Sun Tzu and dating back over 1500 years) says that working modulo $N = pq$ is essentially the same as working modulo p and modulo q at the same time.[2]

Here is an example. Consider all the values modulo 35. They are in one-to-one correspondence with values modulo 5 and modulo 7.

mod5\ mod 7	0	1	2	3	4	5	6
0	0	15	30	10	25	5	20
1	21	1	16	31	11	26	6
2	7	22	2	17	32	12	27
3	28	8	23	3	18	33	13
4	14	29	9	24	4	19	34

Observe that adding 17 and 29 (underlined in the table) is the same as adding 3 (which is 17 mod 7) and 1 (which is 29 mod 7) modulo 7 to get 4; adding 2 (which is 17 mod 5) and 4 (which is 29 mod 5) modulo 5 to get 1; and then looking up the value corresponding to row 1 (modulo 5) and column 4 (modulo 7) in the table to get 11 (in a box in the table). Thus, we can do addition coordinatewise. Same for multiplication.

We now formally state the observations above, generalized to p and q instead of 5 and 7. The proof follows after a few additional remarks.

Theorem 3.2 (Chinese Remainder Theorem)
Let $p \neq q$ be primes, $N = pq$. For each $a \in \mathbb{Z}_p$, $b \in \mathbb{Z}_q$, there is a unique integer c such that $0 \leq c < N$, $c \equiv a \bmod p$, and $c \equiv b \bmod q$. We denote this number $c = \mathrm{crt}(a, b)$. Moreover, given p and q computing $(a, b) \mapsto \mathrm{crt}(a, b)$ can be done in PPT.

To go from c to (a, b), we just compute $(a, b) = (c \bmod p,\ c \bmod q)$. In particular, $0 = \mathrm{crt}(0, 0)$ and $1 = \mathrm{crt}(1, 1)$. To go the other way, we compute $c = \mathrm{crt}(a, b)$ from (a, b) as guaranteed by Theorem 3.2. For general integers $a, b \in \mathbb{Z}$, we can extend the definition by first reducing a modulo p and b modulo q: $\mathrm{crt}(a, b) := \mathrm{crt}([a \bmod p], [b \bmod q])$.

Before we prove the Chinese Remainder Theorem, let's see how we can use CRT to do arithmetic over $\mathbb{Z}_N$ by working modulo p and q separately and then reconstructing the value modulo N when it is needed.

Proposition 3.4. *Let $N = pq$, $c_1 = \mathrm{crt}(a_1, b_1)$, and $c_2 = \mathrm{crt}(a_2, b_2)$.*

$$c_1 + c_2 \equiv \mathrm{crt}(a_1 + a_2,\ b_1 + b_2) \bmod N.$$

$$c_1 \cdot c_2 \equiv \mathrm{crt}(a_1 \cdot a_2,\ b_1 \cdot b_2) \bmod N.$$

Proof. Exercise 3.2 □

[1] This whole section is adapted from notes by Leo Reyzin.
[2] For those comfortable with abstract algebra, the ring $\mathbb{Z}_N$ is isomorphic to the product ring $\mathbb{Z}_p \times \mathbb{Z}_q$.

Proof of Theorem 3.2. First, we will show that for every a and b, we can find a number c satisfying the properties in the theorem. Let $r = [p^{-1} \bmod q]$ and $s = [q^{-1} \bmod p]$. Let

$$c' = rpb + sqa.$$

Reducing modulo p:

$$c' \equiv rpb + sqa \equiv r \cdot 0 \cdot b + 1 \cdot a \equiv a \bmod p.$$

Reducing modulo q:

$$c' \equiv rpb + sqa \equiv 1 \cdot b + s \cdot 0 \cdot a \equiv b \bmod q.$$

Therefore $c' \equiv a \bmod p$ and $c' \equiv b \bmod q$. We're not done, because c' may not be in the appropriate range.

Let $c = [c' \bmod N]$. By definition, $0 \leq c < N$.

$$N \Big| (c - c') \implies p \Big| (c - c') \implies c \equiv c' \equiv a \bmod p.$$

Similarly, $c \equiv b \bmod q$. Hence, c satisfies all the conditions: $0 \leq c < N$, and $c \equiv a \bmod p$, and $c \equiv b \bmod q$.

Second, we show that c is unique. A fixed $c \in \mathbb{Z}_N$ gives exactly one pair $([c \bmod p], [c \bmod q])$. So for $(a, b) \neq (a', b')$, we have that $\mathrm{crt}(a, b) \neq \mathrm{crt}(a', b')$. There are $|\mathbb{Z}_p| \cdot |\mathbb{Z}_q| = pq = N$ possible pairs (a, b), each with at least one value $\mathrm{crt}(a, b) \in \mathbb{Z}_N$ by existence. There are only $|\mathbb{Z}_N| = N$ possible values of c. Thus each pair $(a, b) \in \mathbb{Z}_p \times \mathbb{Z}_q$ has exactly one corresponding $c \in \mathbb{Z}_N$.

Finally, the construction is efficient. It requires computing a constant number of multiplicative inverses, modular reductions, additions and multiplications. Each operation is PPT. □

Example 3.3. Let $p = 7$, $q = 11$, and $N = 77$. To compute $\mathrm{crt}(a, b)$, first we compute $[11^{-1} \bmod 7] = 2$ and $[7^{-1} \bmod 11] = 8$. Then we take $c = [22a + 56b \bmod 77]$.

For instance, $a = 4$ and $b = 3$: $\mathrm{crt}(4, 3) = [256 \bmod 77] = 25$. Going the other way, we see that $25 \equiv 4 \bmod 7$ and $25 \equiv 3 \bmod 11$.

3.4 Multiplication and inverses modulo $N = pq$ ($\mathbb{Z}_N^*$)

Earlier, we saw that multiplication can behave strangely:

$$6 \cdot 10 \equiv 60 \equiv 0 \bmod 15.$$

Moreover, the zero product property and cancellation properties don't hold true modulo N. For example,

$$3 \cdot 5 \equiv 3 \cdot 10 \bmod 15.$$

CRT helps us understand what's going on. Take $p = 3$, $q = 5$, and $N = pq = 15$.

Let's look at the CRT remainders modulo p and q. For the first example, we see $6 = \mathrm{crt}(0, 1)$ and $10 = \mathrm{crt}(1, 0)$. The product of non-zero elements is equal to zero because, looking at their CRT coordinates modulo p and q, each term is congruent to 0 modulo one of the factors p or q. For the second example, $5 = \mathrm{crt}(2, 0)$ and $10 = \mathrm{crt}(1, 0)$. The common multiplier $3 = \mathrm{crt}(0, 3)$ has a zero coordinate modulo p, so both products are $\mathrm{crt}(0, 0)$. Multiplication by 3 loses the differing coordinates of 5 and 10 modulo p.

3.4.1 Multiplicative inverses

We define $\mathbb{Z}_N^*$ to be the subset of $\mathbb{Z}_N$ with multiplicative inverses modulo N. The following proposition says that a number is invertible modulo N if and only if its CRT coordinates are invertible modulo p and q.

Proposition 3.5. *Let $N = pq$ for primes $p \neq q$.*

$$\mathbb{Z}_N^* = \{\mathrm{crt}(a, b) : a \in \mathbb{Z}_p^*,\ b \in \mathbb{Z}_q^*\}.$$

Moreover, for $c \in \mathbb{Z}_N^*$ with $c = \mathrm{crt}(a, b)$, its inverse is $c^{-1} = \mathrm{crt}(a^{-1},\ b^{-1})$.

Equivalently, $\mathbb{Z}_N^* = \{x \in \mathbb{Z}_N : \gcd(x, N) = 1\}$. This is because a coordinate is zero exactly when the corresponding prime divides x. Since $N = pq$ with distinct primes, $\gcd(x, N) = 1$ exactly when neither p nor q divides x, i.e., both coordinates are nonzero.

Corollary 3.2. $|\mathbb{Z}_N^*| = (p-1)(q-1)$.

Proof of Proposition 3.5. First, we will show that if $a \in \mathbb{Z}_p^*$ and $b \in \mathbb{Z}_q^*$, then $c = \mathrm{crt}(a, b)$ is invertible. Because $a \in \mathbb{Z}_p^*$: a has multiplicative inverse $a^{-1} \bmod p$. Likewise, b has multiplicative inverse $b^{-1} \bmod q$. Let $d = \mathrm{crt}(a^{-1},\ b^{-1})$.

$$c \cdot d \equiv \mathrm{crt}(a \cdot a^{-1},\ b \cdot b^{-1}) \equiv \mathrm{crt}(1, 1) \equiv 1 \bmod N.$$

Thus $c^{-1} = d$ is the multiplicative inverse of c modulo N.

Second, we will show the converse: if $c = \mathrm{crt}(a, b)$ has a multiplicative inverse c^{-1} modulo N, then $a \in \mathbb{Z}_p^*$ and $b \in \mathbb{Z}_q^*$.

$$\mathrm{crt}(1, 1) \equiv 1 \equiv c \cdot c^{-1} \equiv \mathrm{crt}(a \cdot [c^{-1} \bmod p], b \cdot [c^{-1} \bmod q]) \bmod N.$$

Reducing modulo p, we get

$$a \cdot [c^{-1} \bmod p] \equiv 1 \bmod p.$$

Hence, $[c^{-1} \bmod p]$ is the inverse of a modulo p, and $a \in \mathbb{Z}_p^*$. Applying the symmetric argument to b shows that $b \in \mathbb{Z}_q^*$. □

Proposition 3.5 gives an efficient algorithm for computing its multiplicative inverses modulo N, **but only if you know the factorization of** N. We will state, without proof, that inverses can be computed in PPT even without knowing p and q. This is a consequence the Extend Euclidean Algorithm, which we're not covering.

Proposition 3.6. *Given $m \geq 2$ and $a \in \mathbb{Z}$, it is possible to compute $a^{-1} \bmod m$, if it exists, in time polynomial in $\|m\|$ and $\|a\|$.*

Multiplicative inverses give us a restricted cancellation rule modulo N: we can cancel the invertible elements.

Proposition 3.7 (Cancellation rule for $\mathbb{Z}_N^*$)**.** *Let $c \in \mathbb{Z}_N^*$, and $u, v \in \mathbb{Z}$. If $cu \equiv cv \bmod N$, then $u \equiv v \bmod N$.*

Proof. $u \equiv c^{-1}cu \equiv c^{-1}cv \equiv v \bmod N$. □

3.5 Squares (quadratic residues) and square roots

Some integers are squares (e.g., $1, 4, 9, \ldots$), and some are not. We could define squares as follows:

> **Definition 3.1** (Square number)**.** *An integer $y \in \mathbb{Z}$ is a* square *if there is a solution $x \in \mathbb{Z}$ to the equation $x^2 = y$. A solution x is called a* square root *of y.*

Notice a couple things. Every square actually has two square roots: $\pm x$. By convention, we don't call 0 a square.

In modular arithmetic, the analogue of square numbers are called **quadratic residues**. For any modulus $m \geq 2$, write $\mathbb{Z}_m^* = \{a \in \mathbb{Z}_m : \gcd(a, m) = 1\}$ for the elements with multiplicative inverses modulo m. This extends the notation already used for primes and products of two distinct primes.

> **Definition 3.2** (Quadratic residue, QR_m)**.** *An element $y \in \mathbb{Z}_m^*$ is a* quadratic residue modulo m *if there is a solution $x \in \mathbb{Z}_m^*$ to the equation $x^2 \equiv y \bmod m$. A solution x is called a* square root *of y. The set of all quadratic residues modulo m is denoted QR_m.*

As with the integers: a square may have multiple square roots, and by convention we don't call y a quadratic residue if $\gcd(y, m) > 1$ by requiring $y \in \mathbb{Z}_m^*$ (rather than $\mathbb{Z}_m$).

We start by proving two simple, but useful facts.

Proposition 3.8. $(-1)^2 \equiv 1 \bmod m$

Proof. $(-1)^2 \equiv (m-1)^2 \equiv m^2 - 2m + 1 \equiv 1 \bmod m$. □

Proposition 3.9. *If $y, z \in QR_m$, then $[yz \bmod m] \in QR_m$.*

Proof. By hypothesis, there exist $x, w \in \mathbb{Z}_m^*$ such that $y \equiv x^2$ and $z \equiv w^2 \bmod m$. Thus $yz \equiv (xw)^2 \bmod m$. The product $[yz \bmod m] \in \mathbb{Z}_m^*$, with inverse $[y^{-1}z^{-1} \bmod m]$. □

> **Example 3.4.** Let $p = 11$. Squaring $1, 2, 3, 4, 5$ gives $1, 4, 9, 5, 3$ modulo 11; the other five nonzero inputs are their negatives $(-1, -2, -3, -4, -5)$, which give the same squares. Thus
>
> $$QR_{11} = \{1, 3, 4, 5, 9\}.$$

3.5.1 Modulo prime p

Throughout this subsection, p is an odd prime.[3]

Proposition 3.10. *Every quadratic residue $y \in QR_p$ has exactly two roots, $\pm x$.*

[3] For $p = 2$, $QR_2 = \{1\}$ and 1 has only one square root modulo 2.

Proof. First, there cannot be just one root. If $x^2 \equiv y \bmod p$, then $(-x)^2 \equiv (-1)^2 \cdot y \equiv y \bmod p$. Thus x and $-x$ are both roots. Suppose for contradiction that these are in fact the same root: $x \equiv -x \bmod p$. Then $2x \equiv 0 \bmod p \implies p \mid 2x \implies p \mid x$ because p is odd. This is impossible because $x \not\equiv 0 \bmod p$. Thus x and $-x$ are distinct roots.

Second, there cannot be more than two roots. Suppose $x^2 \equiv z^2 \bmod p$. Then $(x+z)(x-z) \equiv 0 \bmod p$. Because p is prime, either $x + z \equiv 0 \bmod p$ or $x - z \equiv 0 \bmod p$. Thus, $x \equiv \pm z \bmod p$. □

Corollary 3.3. $|QR_p| = \frac{1}{2} \cdot |\mathbb{Z}_p^*| = (p-1)/2$.

The following is a key fact to remember about quadratic residues modulo primes.

Proposition 3.11 (Euler's Criterion)**.** *For odd prime p and $y \in \mathbb{Z}_p^*$,*

$$y \in QR_p \iff y^{\frac{p-1}{2}} \equiv 1 \bmod p.$$

One direction is easy to check: if $y \in QR_p$, let $x \in \mathbb{Z}_p^*$ satisfy $x^2 \equiv y \bmod p$. By Fermat's Little Theorem:

$$y^{(p-1)/2} \equiv (x^2)^{(p-1)/2} \equiv 1 \bmod p.$$

We will not prove the other direction.

Checking quadratic residuosity mod p

By Euler's Criterion, all we have to do is check if $y^{(p-1)/2} \equiv 1 \bmod p$. If so, then $y \in QR_p$. Otherwise, $y \notin QR_p$. This is efficient using repeated squaring.

Sampling quadratic residues mod p

It is easy to uniformly sample from QR_p.

- Sample $x \xleftarrow{\$} \mathbb{Z}_p^*$ uniformly at random.
- Return $y = [x^2 \bmod p]$.

Clearly $y \in QR_p$. It is uniform over QR_p because each residue y has exactly two preimages and is sampled with probability $2/|\mathbb{Z}_p^*| = 1/|QR_p|$.

Taking square roots mod $p \equiv 3 \bmod 4$

One can compute the square roots of $y \in QR_p$ efficiently. We'll prove it only for odd primes congruent to 3 modulo 4. We skip the algorithm for $p \equiv 1 \bmod 4$, which is too complicated for our purposes.[4]

Proposition 3.12. *Let $p \equiv 3 \bmod 4$ be a prime, and $y \in QR_p$. The square roots of y are*

$$x \equiv \pm y^{\frac{p+1}{4}} \bmod p.$$

[4] See `https://en.wikipedia.org/wiki/Tonelli-Shanks_algorithm`.

Proof. Let $p = 4i + 3$. It suffices to show that $x \equiv y^{\frac{p+1}{4}}$ is a square root of y.

$$x^2 \equiv \left(y^{\frac{p+1}{4}}\right)^2 \equiv \left(y^{i+1}\right)^2 \equiv y^{2i+2} = y^{\frac{p+1}{2}} \equiv y^{\frac{p-1}{2}+1} \equiv y \bmod p.$$

The above uses two facts. First, $1 \equiv y^{\frac{p-1}{2}} \bmod p$ because y is a quadratic residue. Second, $\frac{p+1}{4}$ is an integer because $p \equiv 3 \bmod 4$. □

Example 3.5. Let $p = 11$. For $y = 5$, Euler's Criterion gives $5^5 \equiv 1 \bmod 11$. Thus 5 is a quadratic residue. Because $11 \equiv 3 \bmod 4$, we can compute one of the square roots of 5 using the above formula:

$$x = [5^{(11+1)/4} \bmod 11] = [125 \bmod 11] = 4.$$

The other root is $-4 \equiv 7 \bmod 11$.

3.5.2 Modulo composite $N = pq$

Throughout this subsection, p and q are distinct odd primes and $N = pq$.

The Chinese Remainder Theorem gives us a correspondence between the quadratic residues modulo $N = pq$ and the quadratic residues modulo its prime factors. Taking a square root modulo N is equivalent to taking the square roots modulo p and q separately.

Claim 3.1. *Let $a \in \mathbb{Z}_p^*$, $b \in \mathbb{Z}_q^*$, and $y = \mathrm{crt}(a, b) \in \mathbb{Z}_N^*$. Then $y \in QR_N$ if and only if $a \in QR_p$ and $b \in QR_q$.*

Proof. ($\Rightarrow$) Suppose $y \in QR_N$. Let x be a square root of y: $x^2 \equiv y \bmod N$. Let $u \in \mathbb{Z}_p^*$ and $v \in \mathbb{Z}_q^*$ be such that $x = \mathrm{crt}(u, v)$. By CRT, $y \equiv x^2 \equiv \mathrm{crt}(u^2, v^2) = \mathrm{crt}(a, b) \bmod N$. Hence, $a \equiv u^2 \bmod p$ and $b \equiv v^2 \bmod q$.

($\Leftarrow$) Suppose $a \in QR_p$ and $b \in QR_q$. Let u and v be their square roots: $u^2 \equiv a \bmod p$ and $v^2 \equiv b \bmod q$. $y = \mathrm{crt}(a, b) = \mathrm{crt}(u^2, v^2) \equiv \mathrm{crt}(u, v)^2 \bmod N$. Since $\mathrm{crt}(u, v) \in \mathbb{Z}_N^*$, this shows that $y \in QR_N$. □

Proposition 3.13. *Every quadratic residue $y \in QR_N$ has exactly 4 roots of the form $\mathrm{crt}(\pm u, \pm v)$, where $\pm u$ and $\pm v$ are the square roots of $[y \bmod p]$ and $[y \bmod q]$, respectively.*

Proof. First, all four sign choices are roots by CRT. They are distinct because $u \not\equiv -u \bmod p$ and $v \not\equiv -v \bmod q$, and CRT gives distinct values for distinct coordinate pairs. Second, every root must reduce to u or $-u$ modulo p and to v or $-v$ modulo q. CRT uniqueness rules out any other roots. □

Corollary 3.4. $|QR_N| = \frac{1}{4} \cdot |\mathbb{Z}_N^*| = (p-1)(q-1)/4$.

Sampling quadratic residues mod N

As above, it is easy to uniformly sample from QR_N, given only N.

- Sample $x \xleftarrow{\$} \mathbb{Z}_N^*$ uniformly at random. (Sample $x \xleftarrow{\$} \mathbb{Z}_N$ uniformly and check whether $\gcd(x, N) = 1$; repeat until this holds.[5])
- Return $y = [x^2 \bmod N]$.

The x sampled in Step 1 is uniform over $\mathbb{Z}_N^*$. Each $y \in QR_N$ has exactly four roots, so its probability of being sampled is $4/|\mathbb{Z}_N^*| = 1/|QR_N|$.

Checking quadratic residuosity mod N

If you know the prime factors p, q of N, checking quadratic residuosity is easy. By Claim 3.1, one need only check whether $[y \bmod p] \in QR_p$ and $[y \bmod q] \in QR_q$. First check that $\gcd(y, N) = 1$, then apply Euler's Criterion modulo both primes:

$$y^{(p-1)/2} \equiv 1 \bmod p \quad \text{and} \quad y^{(q-1)/2} \equiv 1 \bmod q.$$

Both checks take PPT using repeated squaring.

Taking square roots modulo N, with known factorization

If you know the prime factors p, q of N, checking whether $y \in QR_N$ and computing its square roots is PPT. First check whether $y \in QR_N$ as above. For $p \equiv q \equiv 3 \bmod 4$, compute one root modulo each prime using the square-root formula:

$$u = [y^{(p+1)/4} \bmod p], \qquad v = [y^{(q+1)/4} \bmod q].$$

The four roots are $\text{crt}(\pm u, \ \pm v)$. Repeated squaring and CRT compute them in PPT. For other odd primes, use a prime square-root algorithm as mentioned earlier.

Example 3.6. Let $p = 7$, $q = 11$, $N = 77$

(a) $y = 9$.

Modulo 7: $y \equiv 2 \bmod 7$. It is a quadratic residue mod 7 because $2^{(7-1)/2} \equiv 1 \bmod 7$. Its roots are $\pm 2^{(7+1)/4} \equiv \pm 4$; that is 3 and 4.

Modulo 11: $y \equiv 9 \bmod 11$. It is a quadratic residue mod 11 because $9^{(11-1)/2} \equiv 1 \bmod 11$. Its roots are $\pm 9^{(11+1)/4} \equiv \pm 3$; that is 3 and 8.

Because 9 is a quadratic residue modulo both p and q, $y \in QR_{77}$. Computing $\text{crt}(\pm 4, \pm 3)$, we can compute the four square roots:

$u \pmod 7$	$v \pmod{11}$	$\text{crt}(u, v)$
3	3	3
3	8	52
4	3	25
4	8	74

[5] As in chapter 2, we suppress negligible sampling failures. A trial succeeds with probability $(1 - 1/p)(1 - 1/q) \geq 4/9$. Capping the number of trials at L^2, where $L = \lceil \log_2 N \rceil$, gives failure probability at most $(5/9)^{L^2}$ and polynomial worst-case runtime. Conditional on success, x remains uniform. Sampling from $\mathbb{Z}_N$ can also use capped rejection of out-of-range L-bit strings.

(b) $y = 2$.

Modulo 7: $y \equiv 2 \bmod 7$. It is a quadratic residue mod 7 because $2^{(7-1)/2} \equiv 1 \bmod 7$. Its roots are $\pm 2^{(7+1)/4} \equiv \pm 4$; that is 3 and 4.

Modulo 11: $y \equiv 2 \bmod 11$. It is not a quadratic residue mod 11 because $2^{(11-1)/2} = 32 \equiv -1 \bmod 11$. It therefore has no square roots modulo 11.

Because 2 is a quadratic residue modulo p but not modulo q, $y \notin QR_{77}$. There are no square roots of 2 modulo 77: any such root would also be a root modulo 11.

3.6 The hardness of inverting squaring mod N (with unknown factorization)

On the other hand: without the factorization of N, there is no known efficient algorithm for computing square roots for random Blum integers. This is no coincidence: **computing square roots modulo N is as hard as factoring N**! The rest of this chapter makes this claim precise.

We will frame the hardness of computing square roots as the hardness of **inverting the squaring function** modulo N:

$$\mathsf{sqr}_N : x \mapsto [x^2 \bmod N].$$

Observe that there are 4 square-roots of $\mathsf{sqr}_N(x)$. An algorithm successfully inverts if it finds *any* square root.

As before we need to specify a distribution over problem instances. A problem instance is defined by the composite modulus N and an input $x \in \mathbb{Z}_N^*$. Since we want to connect this to the hardness of factoring, N is distributed as in the FACTORING assumption: a uniformly random product of two distinct n-bit primes congruent to 3 mod 4. The element x is sampled uniformly over $\mathbb{Z}_N^*$.

Definition 3.3 (Inverting the squaring function)**.** *For an algorithm $\mathcal{A}$, its* inversion advantage *for* squaring modulo Blum integers *is the following probability:*

$$\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n) := \Pr\left[\hat{x}^2 \equiv y \bmod N\right],$$

in the probability experiment $(p, q, N) \leftarrow \mathsf{Blum}(1^n)$, $x \xleftarrow{\$} \mathbb{Z}_N^*$, $y := \mathsf{sqr}_N(x)$, $\hat{x} \leftarrow \mathcal{A}(N, y)$.

Let's recall the factoring assumption from the previous chapter, following the notation above.

Definition 3.4 (Factoring Problem)**.** *For an algorithm $\mathcal{A}$, its* factoring advantage *is the following probability:*

$$\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{A}}(n) := \Pr\left[(1 < d < N) \text{ and } d \mid N\right],$$

in the probability experiment $(p, q, N) \leftarrow \mathsf{Blum}(1^n)$, $d \leftarrow \mathcal{A}(N)$.

Assumption 3.1 (Factoring Blum Integers (FACTORING))
For every PPT algorithm $\mathcal{A}$, $\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{A}}(n)$ is negligible. Namely, there exists a negligible function negl *such that for all sufficiently large n,*

$$\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{A}}(n) < \mathsf{negl}(n).$$

Now we can state the theorem that we've been building towards: If the Factoring Assumption holds, then inverting the squaring function is hard:

Theorem 3.3
If FACTORING*, then the following holds. For every PPT algorithm $\mathcal{A}$, $\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)$ is negligle. Namely, there exists a negligible function* negl *such that for all sufficiently large n,*

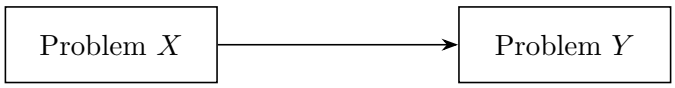
$$\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n) < \mathsf{negl}(n).$$

Like many theorems in this class, this theorem is proved using a *proof by reduction*. In the next section, we'll discuss proofs by reductions and prove a Theorem 3.3.

3.7 Proofs by reduction

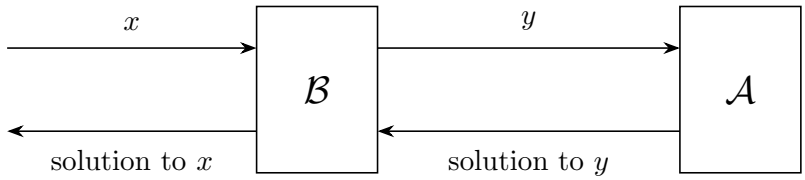
A **reduction from problem X to problem Y** is an algorithm for transforming an algorithm solving Y into an algorithm for solving X. A reduction from X to Y means if you can solve Y in PPT, you can also solve X in PPT. A reduction implies that, for PPT algorithms:

- **Problem X is no harder than Problem Y.**
- **Problem Y is at least as hard as Problem X.**



Given an algorithm $\mathcal{A}$ for Y, the reduction describes an algorithm $\mathcal{B}$ for X. Algorithm $\mathcal{B}$ makes one or many calls to $\mathcal{A}$ as a subroutine. Notice the types of the inputs and outputs:

- $\mathcal{B}$'s input: an instance x of problem X
- $\mathcal{A}$'s input: one or many instances y of problem Y (chosen by $\mathcal{B}$ as a function of x)
- $\mathcal{A}$'s output: solution(s) to y
- $\mathcal{B}$'s output: solution to x



A reduction is **correct** if the following holds: for *any* algorithm $\mathcal{A}$ correctly solving problem Y, the algorithm $\mathcal{B}$ (which depends on $\mathcal{A}$) correctly solves problem X. This is a strong requirement: one cannot make any assumptions about how $\mathcal{A}$ works—just that it does work. The meaning of "solve"—on which inputs and with what probability—depends on context.

A reduction is **efficient** if $\mathcal{B}$ is PPT whenever $\mathcal{A}$ is PPT. With an efficient reduction: if you can solve Y efficiently, then you can also solve X efficiently. This requires the number subroutine calls to $\mathcal{A}$ to be polynomially-bounded, and the additional computation performed by $\mathcal{B}$ to be PPT.

Example 3.7 (Reduction from 2-SUM to Set Intersection)**.** Consider the following two problems:

- **2-SUM:** Given a finite set $S \subseteq \mathbb{Z}$ and an integer t, output elements $a, b \in S$ such that $a + b = t$, or $\perp$ if no such pair exists.
- **Set Intersection:** Given finite sets $U, V \subseteq \mathbb{Z}$, output an element of $U \cap V$, or $\perp$ if the sets are disjoint.

Here is a reduction from 2-SUM to Set Intersection. Given an algorithm $\mathcal{A}$ for Set Intersection, we construct an algorithm $\mathcal{B}$ for 2-SUM:

$\mathcal{B}(S, t)$

$U := S$
$V := \{t - b : b \in S\}$
$a \leftarrow \mathcal{A}(U, V)$
if $a = \perp$ **then return** $\perp$
return $(a,\ t - a)$

For example, let $S = \{1, 4, 5, 7, 9\}$ and $t = 10$. Then

$$U = \{1, 4, 5, 7, 9\}, \qquad V = \{9, 6, 5, 3, 1\}, \qquad U \cap V = \{1, 5, 9\}.$$

If $\mathcal{A}$ returns 1, then $\mathcal{B}$ returns $(1, 9)$. If $\mathcal{A}$ returns 9, then $\mathcal{B}$ returns $(9, 1)$. If $\mathcal{A}$ returns 5, then $\mathcal{B}$ returns $(5, 5)$. Every answer is correct; the reduction works regardless of which intersection element $\mathcal{A}$ chooses.

$\mathcal{B}$ is correct: Suppose $\mathcal{A}$ correctly solves Set Intersection. To prove that $\mathcal{B}$ correctly solved 2-SUM, we show two things: (i) If a valid 2-SUM solution (a, b) exists, then $\mathcal{B}$ returns a valid 2-SUM solution. (ii) If there is no 2-SUM solution, then $\mathcal{B}$ returns $\perp$.

Part (i): If elements $a, b \in S$ satisfy $a + b = t$, then $a \in U$ and $t - b = a \in V$. Therefore $U \cap V$ is nonempty, and $\mathcal{A}$ returns an element $a' \in U \cap V$. Then $a' \in S$ and $a' = t - b'$ for some $b' \in S$. The returned pair (a', b') solves 2-SUM: $a' + b' = t$.

Part (ii): We prove the contrapositive. If $\mathcal{B}(S) \neq \perp$, then $\mathcal{A}(U, V) \neq \perp$. We've already shown that if $\mathcal{A}$ returns an element $a' \in U \cap V$ then $\mathcal{B}$ returns a valid output. Hence a valid 2-SUM solution exists.

$\mathcal{B}$ is efficient: The algorithm $\mathcal{B}$ makes one call to $\mathcal{A}$, on U and V. Constructing V takes $O(|S|)$ arithmetic operations whose complexity is poly in the length of the input integers. Thus the construction and the size of the subroutine input are polynomial in the total binary input length of (S, t). If $\mathcal{A}$ is PPT, then $\mathcal{B}$ is PPT.

3.7.1 Hardness reductions

Suppose X is a problem conjectured to be hard, like factoring. An efficient reduction from X to Y means that **Y is as hard as X** (for PPT algorithms). "Hardness flows in the direction of the reduction."[6] This makes reductions very useful for cryptography. To show that an encryption scheme is secure if factoring is hard, we have to show a reduction from factoring to breaking the encryption scheme. If we do, breaking the encryption scheme is as hard as factoring.

For Theorem 3.3, X is factoring N and Y is computing square roots modulo N. The proof by reduction is structured as follows.

- Let $\mathcal{A}$ be a PPT algorithm for the Square-roots Problem. Assume for contradiction that $\mathcal{A}$ has non-negligible advantage $\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)$.
- Using $\mathcal{A}$ as a subroutine, construct a different PPT algorithm $\mathcal{B}$ for the Factoring Problem with advantage $\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{B}}(n)$.
- Prove the following:

$$\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n) \text{ is non-negligible } \implies \mathbf{Adv}^{\mathsf{factor}}_{\mathcal{B}}(n) \text{ is non-negligible.}$$

  To do so, show that the following holds for some polynomial poly:

$$\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{B}}(n) \geq \frac{\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)}{\mathsf{poly}(n)}. \tag{3.1}$$

- This contradicts the FACTORING assumption. Conclude that:

$$\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n) < \mathsf{negl}(n).$$

Proof of Theorem 3.3

Proof. Suppose for contradiction that $\mathcal{A}$ is an PPT algorithm for inversting squaring modulo Blum integers with non-negligible advantage $\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)$.

We construct a PPT algorithm $\mathcal{B}$ for factoring Blum integers. On input N, $\mathcal{B}(N)$ does the following:

[6] I learned this from Gabe Schoenbach.

Factoring using a square-root oracle: $\mathcal{B}(N)$

1 $x \xleftarrow{\$} \mathbb{Z}_N^*$; $y \leftarrow \mathsf{sqr}_N(x)$
2 $\hat{x} \leftarrow \mathcal{A}(N, y)$ // *Noninteger outputs of $\mathcal{A}$ are interpreted as $\perp$.*
3 **if** $\hat{x}^2 \not\equiv y \bmod N$ **then return** $\perp$
4 **if** $\hat{x} \equiv \pm x \bmod N$ **then return** $\perp$
5 $\hat{p} \leftarrow \mathsf{gcd}(x + \hat{x}, N)$
6 **return** $\hat{p}$

Observe that if $\mathcal{A}$ is PPT then $\mathcal{B}$ is PPT. This is because each line of $\mathcal{B}$'s code can be computed in polynomial time as shown earlier in these notes.

We'll prove the theorem in two steps. First, $\mathcal{B}$ doesn't abort too often. Second, when $\mathcal{B}$ doesn't abort, it factors N.

Step 1: We will show that $\Pr[\mathcal{B}(N) \neq \perp]$ is non-negligible. We need to compute

$$\begin{aligned}\Pr\big[\mathcal{B}(N) \neq \perp\big] &= \Pr\big[\underbrace{\hat{x}^2 \equiv y \bmod N}_{\text{line 3}} \text{ AND } \underbrace{\hat{x} \not\equiv \pm x \bmod N}_{\text{line 4}}\big] \\ &= \Pr\left[\hat{x}^2 \equiv y \bmod N\right] \cdot \Pr\left[\hat{x} \not\equiv \pm x \bmod N \;\middle|\; \hat{x}^2 \equiv y \bmod N\right].\end{aligned}$$

When run by $\mathcal{B}$ on Line 2, $\mathcal{A}$'s input (N, y) is distributed as in the problem of inverting squaring: $N \xleftarrow{\$} \mathsf{Blum}(1^n)$ and y uniform over QR_N. By assumption on $\mathcal{A}$:

$$\Pr[\hat{x}^2 \equiv y \bmod N] = \mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n).$$

Every quadratic residue y has exactly 4 roots. The root x was sampled uniformly at random among the roots of y. Therefore, not matter how $\mathcal{A}$ selects the root $\hat{x}$, the probability that $\hat{x} \equiv \pm x \bmod N$ is exactly $1/2$. Hence,

$$\Pr\left[\hat{x} \not\equiv \pm x \bmod N \;\middle|\; \hat{x}^2 \equiv y \bmod N\right] = \frac{1}{2}.$$

Combining the above equations gives

$$\Pr\big[\mathcal{B}(N) \neq \perp\big] = \frac{\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)}{2},$$

which is non-negligible.

Step 2: We will show that whenever $\mathcal{B}(N) \neq \perp$, it returns a non-trivial divisor of N. When $\mathcal{B}(N) \neq \perp$, we have two square roots x and $\hat{x}$ of y such that $x \not\equiv \pm\hat{x}$. We must show that $\hat{p} = \gcd(x + \hat{x},\ N)$ is a non-trivial divisor of N.

We use the CRT representation of x and $\hat{x}$ modulo p and q. Let $a \in \mathbb{Z}_p$ and $b \in \mathbb{Z}_q$ such that $x = \mathrm{crt}(a, b)$. Observe that because $x \in \mathbb{Z}_N^*$, we have $a \neq 0$ and $b \neq 0$.

Because $\hat{x}^2 \equiv x^2 \bmod N$, we know that $\hat{x} \equiv \mathrm{crt}(\pm a,\ \pm b) \pmod N$. Because $-1 \equiv \mathrm{crt}(-1, -1) \pmod N$ and $\hat{x} \not\equiv \pm x \bmod N$, we know $\hat{x}$ must be congruent modulo N to $\mathrm{crt}(-a, b)$ or $\mathrm{crt}(a, -b)$. Hence

$$x + \hat{x} \equiv \mathrm{crt}(0, 2b) \bmod N \quad \text{or} \quad x + \hat{x} \equiv \mathrm{crt}(2a, 0) \bmod N.$$

Because p and q are odd, $2a \not\equiv 0 \bmod p$ and $2b \not\equiv 0 \bmod q$. By the formula for computing crt, $x + \hat{x}$ is thus a multiple of exactly one of p and q. Therefore $\gcd(x + \hat{x},\ N)$ is either p or q.

Putting it all together: By the above,

$$\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{B}}(n) = \Pr[\mathcal{B}(N) \neq \perp] = \frac{\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)}{2}.$$

If $\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)$ is non-negligible, then $\mathbf{Adv}^{\mathsf{factor}}_{\mathcal{B}}(n)$ is also non-negligible. This contradicts the Factoring assumption. Therefore $\mathbf{Adv}^{\mathsf{inv}}_{\mathsf{sqr},\mathcal{A}}(n)$ is negligible. □

3.8 Exercises

Exercise 3.1. Prove Proposition 3.1.

Exercise 3.2. Prove Proposition 3.4.

Exercise 3.3. Let $p = 5$, $q = 13$, and $N = pq$.

(a) Compute $5^{-1} \bmod 13$

(b) Compute $13^{-1} \bmod 5$

(c) Using the formula for CRT in the proof of Theorem 3.2, compute the value $\mathrm{crt}(2, 6)$ modulo N.

Note: You do not have to memorize the CRT formula.

Exercise 3.4. List the quadratic residues and non-residues modulo 17.

Exercise 3.5. Let $p = 5$, $q = 13$, and $N = pq$. Find the four square roots of 1 modulo N. Which roots are themselves quadratic residues?

Hint: Reuse your work from Exercise 3.3.

Exercise 3.6. Let $p = 47$, $q = 7$, and $N = pq = 329$.

(a) Is $2 \in QR_{47}$? If so, what are its square roots?

(b) Is $40 \in QR_{47}$? If so, what are its square roots?

Hint: The easiest way I see to compute this by hand is to simplify using the following: $40^2 \equiv (-7)^2 \equiv 2 \bmod 47$.

(c) Is $2 \in QR_{329}$? If so, what are its square roots?

(d) Is $40 \in QR_{329}$? If so, what are its square roots?

Exercise 3.7. Find all solutions $x \in \mathbb{Z}_{21}$ to the equation $x^2 \equiv 9 \bmod 21$. Why doesn't this contradict the claim that every $y \in QR_N$ has exactly four square roots, for $N = pq$ the product of distinct odd primes?

Exercise 3.8. Let $p, q \equiv 3 \bmod 4$ be distinct primes, and $N = pq$. Prove the following

(a) $-1 \notin QR_p$.

(b) If $y \in QR_p$, then $-y \notin QR_p$.

(c) For $y \in QR_n$: exactly one square root of y is itself a quadratic residue.
(Notice how this result contrasts with what you found in Exercise 3.5.)

Exercise 3.9. Let $N = pq$ be the product of distinct odd primes. Let x and z satisfy the following:

- $x^2 \equiv z^2 \bmod N$
- $x \not\equiv \pm z \bmod N$

Show that $\gcd(x + z,\ N)$ is equal to p or q.